Unreal Engine MCP Python Bridge Plugin

This is a plugin for Unreal Engine (UE) that creates a server implementation of Model Context Protocol (MCP). This allows MCP clients, like Anthropic's Claude, to access the full UE Python API.

Use Cases

- Developing tools and workflows in Python with an agent like Claude.
- Intelligent and dynamic automation of such workflows.
- General collaborative development with an agent.

Prerequisites

- Visual Studio 2019 or higher.
- An AI Agent. Below, we assume Claude will be used. But any AI Agent that implements MCP should suffice.
- Unreal Engine 5 with the Python Editor Script Plugin enabled.
- Note the Unreal Engine Python API.

Installing from GitHub

1. Create a new Unreal Engine C++ project.
2. Under the project root directory, find the `Plugins` folder.
3. Clone the UnrealMCPBridge repo into the `Plugins` folder so that there is a new containing folder with all the project contents underneath.
4. Right-click your UE project file (ends with `.uproject`) and select "Generate Visual Studio project files". If you don't immediately see that option, first select "Show more options" and it should appear.
5. Open your new Visual Studio project and build.

Installing from Fab

1. After purchasing the plugin on Fab, find the plugin in your Library in the Epic Games Launcher.
2. Click "Install to Engine" and choose the appropriate version of Unreal Engine.

Configure Claude for Desktop

1. Copy `unreal_mcp_client.py` from the MCPClient folder on GitHub to a location of your choice.

2. Find your `claude_desktop_config.json` configuration file. Mac location: `~/Library/Application\ Support/Claude/claude_desktop_config.json` Windows location: `[path_to_your_user_account]\AppData\Roaming\Claude\claude_desktop_config.json`

3. Add the `unreal-engine` server section to your config file and update the path location excluding the square brackets, below.

    - Mac path format: `/[path_from_step_1]/unreal_mcp_client.py`
    - Windows path format: `C:\\[path_from_step_1]\\unreal_mcp_client.py`

```
{
  "mcpServers": {
    "unreal-engine": {
      "command": "python",
      "args": [
        "[mac_or_windows_format_path_to_unreal_mcp_client.py]"
      ]
    }
  }
}
```

4. Create a new Unreal Engine project or load an existing project.
5. From the "Edit" menu, select "Plugins".
6. Enable these plugins: `UnrealMCPBridge` and `Python Editor Script Plugin`.
7. Restart Unreal Engine.
8. There should be a new toolbar button that says, "Start MCP Bridge" when you hover with your mouse.
9. Click the MCP Bridge button. A pop-up will state, "MCP Python bridge running." The Output Log will note a new socket server listening on 127.0.0.1:9000.
10. Launch Claude as an administrator.
11. Click the "Attach from MCP" plug-icon. Under "Choose an integration" are two test Prompts: `create_castle` and `create_town`. You can edit their implementations in `unreal_server_init.py` under the `Content` folder of the plugin. Be sure to restart Unreal Engine after any changes.
12. Alternatively, ask Claude to build in your project using the UE Python API.
13. A list of currently implemented tools can be found by clicking the hammer-icon to the left of the plug-icon.

Configure Claude Code

Claude Code is Anthropic's CLI tool for agentic coding. It supports MCP servers via a `.mcp.json` configuration file in your project root.

Option A: CLI Command (Recommended)

From your Unreal Engine project root directory, run:

Mac/Linux:

```
claude mcp add --transport stdio --scope project unreal-bridge -- python
/path/to/unreal_mcp_client.py
```

Windows:

```
claude mcp add --transport stdio --scope project unreal-bridge -- python
C:\path\to\unreal_mcp_client.py
```

This creates a `.mcp.json` file in your project root automatically.

Option B: Manual Configuration

1. Copy `unreal_mcp_client.py` from the MCPClient folder to a location of your choice.

2. Create a `.mcp.json` file in your Unreal Engine project root directory (the folder containing your `.uproject` file) with the following contents:

Mac/Linux:

```
{
  "mcpServers": {
    "unreal-bridge": {
      "command": "python",
      "args": ["/path/to/unreal_mcp_client.py"],
      "env": {}
    }
  }
}
```

Windows (use forward slashes or escaped backslashes):

```
{
  "mcpServers": {
    "unreal-bridge": {
      "command": "python",
      "args": ["C:/path/to/unreal_mcp_client.py"],
      "env": {}
    }
  }
}
```

Starting a Session

1. Open your Unreal Engine project and click the MCP Bridge toolbar button to start the server.
2. Open a terminal in your project root directory and launch Claude Code:

```
claude
```

3. Claude Code will detect the `.mcp.json` and prompt you to approve the MCP server on first use.
4. Once approved, all MCP tools (get_actors, spawn_actor, execute_python, etc.) are available. Claude Code can call them directly during the conversation.

Verifying the Connection

Inside a Claude Code session, type `/mcp` to check the status of connected MCP servers. You should see the `unreal-bridge` server listed as connected.

Available Tools

These tools are defined in `unreal_mcp_client.py` and are available to the AI agent once the MCP bridge is connected. In Claude Desktop they appear under the hammer-icon. In Claude Code they can be called directly during conversation.

Project & Asset Discovery

Tool	Parameters	Description
`get_project_dir`	—	Returns the top-level project directory path (e.g., `D:/MyProject/`).
`get_content_dir`	—	Returns the Content directory path (e.g., `D:/MyProject/Content/`).
`find_basic_shapes`	—	Returns a list of built-in basic shapes (cube, sphere, cylinder, etc.) that can be used for building.
`find_assets`	`asset_name`	Searches the asset registry for assets matching a name. Use keywords like `Floor`, `Wall`, `Door` to discover meshes.
`get_asset`	`asset_path`	Returns the bounding box dimensions of a static mesh asset.

Actor Management

Tool	Parameters	Description
`get_actors`	—	Lists all actors in the current level with their name, class, and location.
`get_actor_details`	`actor_name`	Returns all properties and details for a specific actor by name.
`get_selected_actors`	—	Returns the actors currently selected in the editor viewport. Useful for

Tool	Parameters	Description
		inspecting what the user is looking at.
`spawn_actor`	`asset_path`, `location_x/y/z`, `rotation_x/y/z`, `scale_x/y/z`	Spawns a static mesh actor at the given location, rotation, and scale. All position/rotation/scale parameters default to 0.
`modify_actor`	`actor_name`, `property_name`, `property_value`	Modifies a property of an existing actor. The value is a string that gets converted to the appropriate type on the server side.
`set_material`	`actor_name`, `material_path`	Applies a material to a static mesh actor. Use asset paths like `/Game/Materials/M_MyMaterial`.
`delete_all_static_mesh_actors`	—	Deletes **all** static mesh actors in the scene. Use with caution — this removes everything, not just what was recently placed.

Building Utilities

Tool	Parameters	Description
`create_grid`	`asset_path`, `grid_width`, `grid_length`	Creates a grid of tiles using the given asset, automatically spaced based on the asset's bounding box. `grid_width` and `grid_length` are the number of tiles in each dimension.
`create_town`	`town_center_x/y`, `town_width`, `town_height`	Creates a procedural town layout centered at the given

Tool	Parameters	Description
		position using available assets.

Blueprint Execution

Tool	Parameters	Description
`run_blueprint_function`	`blueprint_name`, `function_name`, `arguments`	Executes a function defined in a Blueprint. Arguments are passed as a comma-separated string.

Viewport & Screenshots

Tool	Parameters	Description
`take_screenshot`	—	Captures the active editor viewport to a PNG file and returns the file path.
`set_viewport_camera`	`location_x/y/z`, `rotation_pitch/yaw/roll`	Moves the editor viewport camera to a specific location and rotation. Pitch = look up/down, Yaw = compass heading, Roll = tilt.
`focus_viewport_on_actor`	`actor_name`, `distance`	Points the camera at a named actor from a 45-degree overhead angle. Distance is auto-calculated from the actor's bounds, or can be overridden (set to 0 for auto).

Performance Tracing & Analysis

Tool	Parameters	Description
`start_trace`	—	Starts capturing an Unreal Insights trace to file. Captures CPU, GPU, Frame, Counters,

Tool	Parameters	Description
		and Region channels. Call `stop_trace` when done.
`stop_trace`	—	Stops the current trace capture. Returns the absolute path to the `.utrace` file.
`analyze_trace`	`trace_path`, `top_n`	Parses a `.utrace` file and returns a JSON summary including: trace duration, thread info, frame statistics (avg/min/max FPS), and the top N most expensive timing scopes sorted by total inclusive time. Each scope includes name, source file/line, call count, and total/avg/max inclusive time in milliseconds. Default `top_n` = 50.
`get_trace_spikes`	`trace_path`, `budget_ms`, `max_frames`, `top_scopes_per_frame`	Finds frames that exceeded a time budget. For each spike frame, returns the top timing scopes contributing to that frame. Default budget = 33.33ms (30fps).
`get_trace_frame_summary`	`trace_path`	Returns frame timing statistics from a `.utrace` file: frame count, avg/min/max frame time, FPS, percentiles (p50/p90/p95/p99), and counts of frames exceeding 60fps and 30fps budgets.

Python Execution

Tool	Parameters	Description
`execute_python`	`code`	Executes arbitrary Python code inside the Unreal Engine process. This gives the agent access to the full Unreal Engine Python API. Returns printed output and error tracebacks.

`execute_python` is the most powerful tool. The agent can import `unreal`, call any editor subsystem, manipulate assets, modify materials, create Blueprints, and run any logic that the UE Python API supports. The other tools are convenience wrappers for common operations that don't require writing Python code.

Available Prompts

Prompts are pre-written instructions that guide the agent through a multi-step workflow. In Claude Desktop, they appear under the plug-icon.

Prompt	Description
`create_castle`	Clears the scene, finds basic shapes, and builds a castle from primitives.
`create_town`	Clears the scene, finds a floor tile, builds a grid, and generates a town layout.

You can add your own prompts by following the pattern in the "Developing New Tools and Prompts" section below.

C++ Libraries (via execute_python)

The plugin bundles four C++ `BlueprintFunctionLibrary` classes that auto-reflect to Python when the plugin is loaded. These are accessed via `execute_python` using `import unreal`:

Library	Python Access	Purpose
`BlueprintGraphLibrary`	`unreal.BlueprintGraphLibrary`	Create Blueprint assets, add event/function/variable nodes, wire exec and data pins, compile and save.
`NiagaraEditorLibrary`	`unreal.NiagaraEditorLibrary`	Create Niagara particle systems, add emitters and modules, set parameters, configure renderers (Sprite/Ribbon/Light/Mesh/Decal), compile.
`ViewportCaptureLibrary`	`unreal.ViewportCaptureLibrary`	Synchronous viewport screenshot capture via ReadPixels. Used internally by `take_screenshot`.
`TraceAnalysisLibrary`	`unreal.TraceAnalysisLibrary`	Parse Unreal Insights `.utrace` files — extract timing scopes, frame stats, thread info. Used internally by `analyze_trace`, `get_trace_spikes`, `get_trace_frame_summary`.

Developing New Tools and Prompts

Examine `unreal_mcp_client.py` and you'll see how MCP defines tools and prompts using Python decorators above functions. As an example:

```
@mcp.tool()
def get_project_dir() -> str:
    """Get the top level project directory"""
    result = send_command("get_project_dir")
    if result.get("status") == "success":
        response = result.get("result")
        return response
    else:
        return json.dumps(result)
```

This sends the `get_project_dir` command to Unreal Engine for execution and returns the project level directory for the current project. Under the `Content` folder of the plugin, you will see the server-side implementation of this tool command:

```
@staticmethod
def get_project_dir():
    """Get the top level project directory"""
    try:
        project_dir = unreal.Paths.project_dir()
        return json.dumps({
            "status": "success",
            "result" : f"{project_dir}"
        })
    except Exception as e:
        return json.dumps({ "status": "error", "message": str(e) })
```

Follow this pattern to create new tools that appear in the Claude Desktop interface under the hammer-icon, or as callable tools in Claude Code.

Implementing new prompts is a matter of adding them to `unreal_mcp_client.py`. As an example, here is the `create_castle` prompt from `unreal_mcp_client.py`:

```
@mcp.prompt()
def create_castle() -> str:
    """Create a castle"""
    return f"""
Please create a castle in the current Unreal Engine project.
0. Refer to the Unreal Engine Python API when creating new python code:
https://dev.epicgames.com/documentation/en-us/unreal-engine/python-api/?app
lication_version=5.7
1. Clear all the StaticMeshActors in the scene.
2. Get the project directory and the content directory.
3. Find basic shapes to use for building structures.
4. Create a castle using these basic shapes.
"""
```

Be sure to maintain triple-quotes so the entire prompt is returned. A good way to iterate over creating prompts is simply iterating each step number with Claude until you get satisfactory results. Then combine them all into a numbered step-by-step prompt as shown.

You must restart Claude for any changes to `unreal_mcp_client.py` to take effect. Note for Windows, you might need to end the Claude process in the Task Manager to truly restart Claude. For Claude Code, restart the session or use `/mcp` to reconnect.

Creating Blueprints and Niagara Systems

The `execute_python` tool gives the agent access to the full Unreal Engine Python API, including the C++ wrapper libraries bundled with this plugin (see table above).

Creating Blueprints

Ask the agent to create a Blueprint and it will use `execute_python` to call `BlueprintGraphLibrary` functions. The library supports creating Actor Blueprints, adding event nodes (BeginPlay, Tick, custom events), function call nodes, variables, branch logic, and pin connections — including exec pin wiring on event nodes.

Example prompt for Claude Desktop — add this to `unreal_mcp_client.py`:

```
@mcp.prompt()
def create_rotating_actor() -> str:
    """Create a Blueprint actor that rotates continuously"""
    return f"""
Create a Blueprint actor that rotates continuously using the
BlueprintGraphLibrary.
Use execute_python with `import unreal` and `unreal.BlueprintGraphLibrary`
to:
1. Create a new Blueprint at /Game/Blueprints/BP_Rotator with Actor as the
parent class.
2. Add a ReceiveBeginPlay event node.
3. Add a call to SetActorRotation or AddActorLocalRotation.
4. Add a variable for rotation speed (float, instance editable, default
1.0).
5. Connect the event to the function call.
6. Compile and save the Blueprint.
7. Spawn an instance of the Blueprint into the level.
"""
```

Example prompt for Claude Code — type directly in the conversation:

```
Create a Blueprint at /Game/Blueprints/BP_DayNightCycle that has a public
float variable called "CycleSpeed" and a DirectionalLight reference
variable called "SunLight". On BeginPlay it should print "Day/Night cycle
```

```
started" to the screen.
```

Creating Niagara Particle Effects

Ask the agent to create a Niagara system and it will use `execute_python` to call `NiagaraEditorLibrary` functions. The library supports creating systems from scratch, adding emitters with modules (spawn rate, forces, velocity, etc.), configuring renderers (Sprite, Ribbon, Light, Mesh, Decal), and setting parameters.

Example prompt for Claude Desktop — add this to `unreal_mcp_client.py`:

```
@mcp.prompt()
def create_rain() -> str:
    """Create a rain particle effect"""
    return f"""
Create a rain Niagara particle system using the NiagaraEditorLibrary.
Use execute_python with `import unreal` and `unreal.NiagaraEditorLibrary`
to:
1. Create a new Niagara system at /Game/VFX/NS_Rain.
2. Add an empty emitter named "RainDrops".
3. Add these modules in order:
   - SpawnRate (EmitterUpdate) -- set to 500 particles/sec.
   - BoxLocation (ParticleSpawn) -- set Box Size to (3000, 3000, 50).
   - AddVelocity (ParticleSpawn) -- set Velocity to (200, 0, -1500) for
angled rain.
   - GravityForce (ParticleUpdate).
   - SolveForcesAndVelocity (ParticleUpdate).
4. Set Initialize Particle lifetime to 2.0 seconds.
5. Set Initialize Particle sprite size to (1, 15) for thin streaks.
6. Set the sprite renderer to VelocityAligned facing.
7. Compile and save the system.
8. Spawn the system into the level at (0, 0, 1500).
"""
```

Example prompt for Claude Code — type directly in the conversation:

```
Create a Niagara lightning effect with a beam emitter. It should be a
jagged blue-white bolt that fires once from (0, 0, 2000) down to (0, 0, 0).
Use a Ribbon renderer with JitterPosition for the jagged shape.
```

Analyzing Performance with Unreal Insights

The agent can capture and analyze performance traces without leaving the conversation. This uses the `TraceAnalysisLibrary` to parse `.utrace` files and extract function-level timing data — the same data visible in the Unreal Insights standalone application.

Workflow:

1. start_trace → begins capturing CPU/GPU/Frame data to a .utrace file

2. [perform actions] → play in editor, stress test, build geometry

3. stop_trace → stops capture, returns the .utrace file path

4. analyze_trace(path, 20) → returns top 20 most expensive timing scopes with call counts and durations

5. get_trace_spikes(path) → finds frames that exceeded the frame budget and shows what caused each spike

6. get_trace_frame_summary → frame time percentiles (p50/p90/p95/p99), FPS stats, budget violations

Example prompt for Claude Code — type directly in the conversation:

```
Start a performance trace, wait 10 seconds, stop the trace, then analyze
it. Show me the top 20 most expensive functions and any frames that
exceeded the 30fps budget.
```

What the analysis returns:

- **Per-scope data:** Function name, source file and line number, call count, total/average/max inclusive time in milliseconds
- **Frame statistics:** Frame count, avg/min/max frame time, average FPS, p50/p90/p95/p99 percentiles
- **Spike analysis:** Frames exceeding a budget (e.g., 33.33ms for 30fps) with the top contributing scopes per frame
- **Thread info:** All threads with names and group assignments

Tips for Best Results

- **Be specific about asset paths.** Tell the agent where to save the asset (e.g., `/Game/VFX/NS_MyEffect`) so it doesn't have to guess.
- **Describe the visual result.** "Long thin rain streaks falling at an angle" gives better results than "make rain."
- **Iterate.** Ask the agent to create the effect, preview it in the editor, then ask for adjustments ("make the particles bigger", "add more gravity", "change the color to blue").
- **Close the asset editor** before asking the agent to modify an existing Niagara system or Blueprint. The editor caches its own copy and won't reflect external changes made via Python.
- **For performance analysis,** capture traces during representative workloads — PIE play sessions, complex scenes, or stress tests yield more useful data than idle editor traces.